SecretHandle

Mô tả đề tài và Hướng dẫn triển khai

phiên bản 2 — đã điều chỉnh phạm vi và lịch trình

OS-Minted, Destination- and Task-Bound Opaque Secrets for Android Mobile AI Agents

Tài liệu kỹ thuật nội bộ — lập ngày 10/10/2026

Lĩnh vực	An ninh hệ điều hành Android, bảo mật tác nhân AI di động
Đối tượng	Mô hình tham chiếu Python + bản thảo LaTeX (53 trang, chưa có số đo thực)
Mục tiêu nộp	ACM MobiSys 2027, **Emerging Ideas track**: đăng ký abstract 27/11/2026, nộp bài 04/12/2026 (AoE, không gia hạn); hội nghị 21–25/06/2027, TP. Hồ Chí Minh. Phạm vi: vấn đề + mô hình đe dọa + thiết kế + tái hiện tấn công + bằng chứng sơ bộ (chưa đánh giá đầy đủ). Bản full: Plan B (MobiSys 2028 / USENIX / NDSS / IEEE S&P).
Về thiết bị	Nhóm *không* có Google Pixel; môi trường chính là emulator/Cuttlefish chạy ảnh AOSP tự build (D6). Số đo độ trễ/năng lượng và keystore phần cứng chuyển sang Giới hạn/future work.
Mức sẵn sàng	Mô hình tham chiếu chạy được + kế hoạch đánh giá; chưa có kết quả thực

Tài liệu gồm 3 phần: (I) mô tả đề tài, (II) hướng dẫn từng bước hoàn thành, (III) lộ trình, tiêu chí và rủi ro.
Phụ lục D liệt kê các thay đổi so với phiên bản đầu và lý do.

Mục lục

Phần I — Mô tả đề tài **2**

1 Tóm tắt điều hành **2**

2 Đặt vấn đề và động lực **2**
2.1 Vì sao là vấn đề hệ thống, không chỉ là vấn đề mô hình . . . 2
2.2 Bằng chứng thực tế . . . 2
2.3 Vì sao là bài toán di động . . . 3

3 Câu hỏi nghiên cứu **3**

4 Phạm vi triển khai theo bậc ưu tiên (Tier) **3**

5 Điểm mới và khác biệt so với công trình liên quan **4**

6 Mô hình hệ thống và mô hình mối đe dọa **4**
6.1 Ranh giới tin cậy . . . 4
6.2 Principal tác nhân và bề mặt quan sát . . . 5
6.3 Năng lực của kẻ tấn công . . . 5
6.4 Mục tiêu bảo mật . . . 5
6.5 Ngoài phạm vi (non-goals) . . . 6

7 Kiến trúc tổng thể và các thành phần **6**

8 Thuật toán cốt lõi **6**
8.1 Algorithm 1 — Mint tại nguồn có thẩm quyền . . . 6
8.2 Algorithm 2 — ENTER/Resolve . . . 6
8.3 Algorithm 3 — Observation Redactor . . . 7

9 Vòng đời, thu hồi và phục hồi **7**

10 Phân tích hình thức và tính chất bảo mật **7**
10.1 Bản ghi và chuyển trạng thái . . . 8
10.2 Các giả định . . . 8
10.3 Các kết quả hình thức . . . 8

11 Chỉ số và kế hoạch đánh giá **8**
11.1 Các điều kiện và baseline . . . 9
11.2 Chỉ số . . . 9
11.3 Các cuộc tấn công/stress test . . . 9
11.4 Thí nghiệm quyết định (killer experiment) . . . 10
11.5 Cỡ mẫu và ngân sách chạy . . . 10

12 Hiện trạng kho mã và khoảng trống **10**
12.1 Những gì đã có . . . 11
12.2 Những gì còn thiếu (blocking cho kết quả cấp bài báo) . . . 11

Phần II — Hướng dẫn triển khai **13**

13 Nguyên tắc chung và cách tổ chức công việc **13**
13.1 Nguyên tắc . . . 13
13.2 Điều kiện tiên quyết . . . 13
13.3 Hai track, ba bậc, các cổng quyết định (phiên bản MobiSys 2027) . . . 13

14 Giai đoạn 0 — Chuẩn bị, quyết định thiết kế và đạo đức (W1–W2) **14**
14.1 Quyết định thiết kế (đã chốt) . . . 14
14.2 Công việc . . . 14

15 Giai đoạn 1 — Củng cố mô hình tham chiếu Python (W1–W2, song song, tối đa 2 tuần) **15**
15.1 Công việc . . . 15

16 Giai đoạn 2 — Walking skeleton: lát cắt dọc đầu tiên (W3–W5) **15**
16.1 Công việc . . . 15

17 Giai đoạn 3 — Hoàn thiện lõi dịch vụ và resolver (W6–W8, Track A) **16**
17.1 Công việc . . . 16

18 Giai đoạn 4 — Redaction quan sát và đường nhập riêng (W6–W9, Track A) **17**
18.1 Công việc . . . 17

19 Giai đoạn 5 — Ứng dụng test, endpoint, tác nhân và monitor (Track B, rolling W1–W9) **17**
19.1 Công việc . . . 17

20 Giai đoạn 6 — Baseline và ablation (W6–W10) **18**
20.1 Công việc . . . 18

21 Giai đoạn 7 — Chạy thí nghiệm (W10–W13) **18**
21.1 Công việc . . . 18

22 Giai đoạn 8 — Mở rộng bậc T2 (chỉ khi G2 qua; W9–W12) **19**

23 Giai đoạn 9 — Hạng mục bậc T3 và hướng phát triển (ngoài lịch nộp 04/12/2026) **19**

24 Giai đoạn 10 — Phân tích và viết bài (W5–W7 cho bản nộp; W9–W15 cho bản full) **19**
24.1 Công việc . . . 19

25 Giai đoạn 11 — Đóng gói artifact và nộp bài (W6–W7; hạn 04/12/2026) **20**

Phần III — Lộ trình, tiêu chí, rủi ro **21**

26 Lộ trình thời gian **21**
26.1 Plan A — MobiSys 2027 Emerging Ideas (nộp 04/12/2026) . . . 21
26.2 Plan B — bản full có số đo thật, kỳ nộp kế tiếp . . . 21
26.3 Các cổng quyết định . . . 21

27 Bảng tiêu chí hoàn thành tổng thể (Definition of Done) **21**

28 Rủi ro và phương án giảm thiểu **22**

29 Theo dõi tiến độ **23**

30 Phụ lục A — Cấu trúc thư mục và ánh xạ thành phần **23**

31 Phụ lục B — Lệnh thao tác nhanh **23**

32 Phụ lục C — Sơ đồ phụ thuộc công việc **24**

33 Phụ lục D — Nhật ký thay đổi so với phiên bản 1 **25**

Phần I — Mô tả đề tài

1 Tóm tắt điều hành

Các tác nhân AI di động (mobile GUI agent) như AppAgent, Mobile-Agent, AutoDroid thường phải dùng mật khẩu một lần (OTP), thông tin đăng nhập, địa chỉ, số điện thoại hoặc mã thanh toán để hoàn tất công việc. Hiện nay tác nhân lấy các giá trị này bằng cách đọc ảnh chụp màn hình hoặc cây trợ năng (accessibility tree), rồi gõ lại chúng. Chính vì "giá trị đủ rõ để sao chép" nên nó "đủ rõ để bị đánh cắp": một chỉ thị được chèn (prompt injection), một trang web độc hại, một thông báo giả hoặc tầng ghi log đều có thể chuyển quyền hạn làm việc thành hành vi tiết lộ hoặc chuyển hướng bí mật.

Đề tài SecretHandle đề xuất một cơ chế ở cấp hệ điều hành Android cho phép tác nhân dùng bí mật *"dùng được nhưng vô hình"* (usable but invisible). Một dịch vụ đặc quyền *đúc* (mint) một **handle bí mật (secret handle)** — một đối tượng năng lực (capability) mờ, có xác thực — ngay tại **nguồn có thẩm quyền** (authoritative source): đường ống SMS/thông báo, Credential Manager, clipboard, hoặc ví/wallet. Handle mang theo: loại bí mật, định danh tác vụ, một vị từ đích (destination predicate) trên package/origin/field, và vòng đời theo loại. Tác nhân chỉ quan sát handle và phát lệnh `ENTER(handle)` hướng tới một node giao diện; hệ điều hành giải (resolve) giá trị chỉ sau khi xác thực handle, kiểm tra tác vụ và đích, thi hành hạn dùng/số lần dùng/xác nhận, và giữ chỗ (reserve) handle để chống tranh chấp. Sau đó một bộ *redactor* che giá trị đã nhập trong các kênh trợ năng và pixel.

Đóng góp dự kiến (kèm bậc ưu tiên, xem Mục 4): (1) một trừu tượng handle bí mật được đúc tại nguồn OS, không phải bằng cách "cạo" màn hình **[T1]**; (2) một resolver ràng buộc theo đích và tác vụ, chống chuyển hướng do injection **[T1]**; (3) vòng đời theo loại (single-use/TTL) và che quan sát sau khi nhập **[T1]**; xác nhận mỗi lần dùng cho handle cấp thanh toán **[T3]**; (4) một bản thiết kế triển khai AOSP **[T1]**; (5) một giao thức đánh giá chi tiết trên các luồng đăng nhập/2FA (**[T1]**) và checkout (**[T2]**/**[T3]**) với các tác nhân thật.

2 Đặt vấn đề và động lực

2.1 Vì sao là vấn đề hệ thống, không chỉ là vấn đề mô hình

Các tác nhân di động vận hành ứng dụng bằng cách đọc ảnh chụp màn hình hoặc cây trợ năng, rồi tạo thao tác chạm, vuốt, nhập chữ. Giao diện này hấp dẫn vì không cần mỗi nhà phát triển mở API riêng cho tác nhân. Nhưng chính giao diện đó phá vỡ ranh giới bảo mật: nếu một mã hay thông tin đăng nhập đủ rõ để tác nhân sao chép, thì nó cũng đủ rõ để một mô hình bị thao túng, một cloud endpoint, một webpage bị chèn, một thông báo độc hại hoặc một tầng log đánh cắp.

Lọc prompt (prompt filtering) không đủ để bảo đảm bất biến bảo mật: bộ phát hiện có thể bỏ sót một đoạn nhạy cảm; placeholder có thể bị thay vào sai trường; và một mô hình chưa từng đọc giá trị vẫn có thể điều khiển một resolver cục bộ để tiết lộ giá trị tại đích do kẻ tấn công chọn.

2.2 Bằng chứng thực tế

- Comet browser agent đọc OTP từ Gmail và đăng công khai (Brave, 8/2025).
- Nghiên cứu `2510.27140` cho thấy tác nhân di động chuyển tiếp mã Alipay dưới injection.
- Các nghiên cứu GUI cho thấy nội dung nhúng trực quan hoặc nội dung bên thứ ba có thể lái tác nhân tới hành vi xâm phạm quyền riêng tư.

Mỗi mục trên phải được đọc toàn văn và có trích dẫn archival trước khi nộp (xem checklist ở Phần III).

2.3 Vì sao là bài toán di động

Các nguồn có thẩm quyền đều là dịch vụ OS di động: SMS Retriever và User Consent API, việc che OTP trong thông báo nhạy cảm của Android 15, Credential Manager, clipboard và token hóa ví. Tác nhân chạm tới UI qua trợ năng hoặc qua ADB, nên một OTP buộc phải nhập thường bị đọc từ màn hình. Ràng buộc đích dùng định danh package và WebView origin của Android. Trên server thì không có đường ống SMS của OS, không có Credential Manager, không có ví trên thiết bị và không có UI để nhập — cả bài toán lẫn điểm thi hành đều mang tính di động.

3 Câu hỏi nghiên cứu

Đánh giá được tổ chức quanh sáu câu hỏi (RQ). RQ1–RQ4 là câu hỏi chính (primary) để kiểm soát đa giả thuyết.

RQ1 [T1]
Việc đúc tại nguồn OS có giữ giá trị thô khỏi *mọi* quan sát và dấu vết của tác nhân được thiết bị hóa không?

RQ2 [T1]
Ràng buộc đích/tác vụ có ngăn chuyển hướng do injection mà không chặn việc hoàn thành hợp lệ không?

RQ3 [T1]
Vòng đời và giữ chỗ nguyên tử có ngăn replay, vượt hạn và tranh chấp đồng thời không?

RQ4 [T1]
Redaction quan sát có ngăn việc đọc lại sau khi nhập trên cả pixel và cây trợ năng không?

RQ5 [T1]/[T2]
Chi phí độ trễ, bộ nhớ, năng lượng và tương tác người dùng theo từng nguồn/loại handle là bao nhiêu? (T1: độ trễ, bộ nhớ; T2: năng lượng và các nguồn còn lại.)

RQ6 [T2]
Những nhập nhằng định danh, ứng dụng/WebView/bàn phím/OEM không được hỗ trợ nào gây thất bại hoặc buộc dùng chính sách yếu hơn?

4 Phạm vi triển khai theo bậc ưu tiên (Tier)

Phiên bản đầu trải đều công việc trên bốn nguồn, bốn ứng dụng, bốn cơ chế phần cứng/trình duyệt trong 21 tuần cho 2 kỹ sư. Phiên bản này chia phạm vi thành ba bậc để bài báo luôn có một lõi hoàn chỉnh, đo được.

Bậc	Nội dung	Vai trò trong bài
[T1] MVP bắt buộc	Nguồn SMS/thông báo OTP; handle OTP dùng một lần có TTL; ràng buộc package + trường (View gốc) + tác vụ; resolver nguyên tử; đường nhập riêng; redaction trợ năng + ảnh chụp + bề mặt nguồn (thông báo, ứng dụng SMS, màn hình khóa); kho handle trong bộ nhớ (khởi động lại = thu hồi); app A (đăng nhập + OTP) và app B (2FA nhiều trang, preview đối kháng); B0/B1/B3 và 5 ablation; 1 runner tác nhân × 2 mô hình; RQ1–RQ4 và RQ5 cơ bản	Đủ cho tuyên bố chính: OTP dùng được nhưng vô hình với tác nhân và không bị chuyển hướng

Bậc	Nội dung	Vai trò trong bài
[T2] Nên có	Adapter clipboard nhạy cảm; địa chỉ/số điện thoại (app C); B2 (Autofill, OTP Autofill, Credential Manager và các cơ chế che sẵn có của nền tảng); đo năng lượng; tác nhân thứ hai; WebView ở chế độ fail-closed (ghi "unsupported"); RQ5 đầy đủ, RQ6 một phần	Tăng độ phủ và sức thuyết phục; chỉ làm khi cổng G2 qua
[T3] Mở rộng	Credential Manager; ví + Protected Confirmation (app D); xác thực WebView origin (sửa Chromium/WebView); kho bền vững + nhật ký sự cố; đa người dùng/sao lưu; thiết bị OEM/Knox	Chuyển thành Giới hạn/Hướng phát triển; *không* nêu trong đóng góp nếu chưa làm

5 Điểm mới và khác biệt so với công trình liên quan

Công trình	Cách tiếp cận	SecretHandle khác ở đâu
Anonymization-Enhanced Privacy (arXiv 2602.10139)	Placeholder giữ kiểu, giải cục bộ khi `type`, dựa trên bộ phát hiện NER/OCR, giới hạn phiên	Đúc tại nguồn OS (không phụ thuộc recall của bộ phát hiện), có ràng buộc đích, hỗ trợ OTP, bảo vệ sau khi nhập
AutoDroid (MobiCom 2024)	Thay thế phía prompt, ánh xạ ngược *(cần xác minh toàn văn: AutoDroid chủ yếu là tự động hóa tác vụ; xác nhận cơ chế này thuộc AutoDroid hay thuộc công trình khác)*	Không cạo màn hình; handle không thể giả mạo
CaMeL / Dual-LLM	Ẩn giá trị khỏi planner trong ngữ cảnh gọi công cụ	Áp dụng cho đường dẫn dữ liệu UI của OS di động, không chỉ tool-calling
Autofill / Credential Manager	Do ứng dụng khởi xướng, tin ứng dụng	Tác nhân trợ năng không tự gọi được; SecretHandle kiểm soát ở cấp OS và không tin ứng dụng
Cơ chế che sẵn có của Android (`FLAG_SECURE`, che văn bản trường mật khẩu, content sensitivity, che OTP trong thông báo khi chia sẻ màn hình)	Che theo cửa sổ/trường do ứng dụng hoặc người dùng bật; không gắn với tác vụ, đích hay vòng đời	Gắn che với handle, tác vụ và đích; phủ cả cây trợ năng lẫn ảnh chụp cho *principal tác nhân*; cần đo trong B2 để chỉ rõ chúng bao phủ được đến đâu
Google AP2 / Stripe SPT / token Visa-Mastercard	Token hóa thanh toán phía server	Bao phủ OTP, điện thoại, email, địa chỉ, clipboard; thi hành ở cấp OS; bổ trợ chứ không thay thế

6 Mô hình hệ thống và mô hình mối đe dọa

6.1 Ranh giới tin cậy

Vùng tin cậy (TCB): nhân Android, phần liên quan của `system_server`, dịch vụ SecretHandle, các adapter nguồn, resolver và tích hợp input-dispatch, các redactor trợ năng/màn hình, và phép toán keystore dùng để xác thực bản ghi (nếu giữ authenticator, xem quyết định D1). Đường xác nhận thanh toán có thể

đặt UI xác nhận và ủy quyền khóa trong TEE (Android Protected Confirmation là tùy chọn theo thiết bị, phải phát hiện chứ không được giả định) [T3].

Không tin cậy: tiến trình tác nhân, mô hình cục bộ/từ xa, nhà cung cấp mô hình, bộ nhớ tác nhân, ứng dụng thường, nội dung web được hiển thị, thông báo không được nguồn bảo vệ xác thực, và nhãn do ứng dụng cung cấp.

6.2 Principal tác nhân và bề mặt quan sát

Điểm bổ sung quan trọng so với phiên bản 1

(a) Principal tác nhân. Nhiều tác nhân mã nguồn mở (AppAgent, các runner tương thích AndroidWorld) điều khiển thiết bị từ máy chủ qua ADB: chụp bằng `screencap`, đọc cây giao diện bằng `uiautomator`/UiAutomation, gõ bằng `input`. Chủ thể gọi các lệnh này là UID `shell`, không phải một ứng dụng trợ năng. Do đó "principal tác nhân" A phải được định nghĩa cho cả ba kiểu: (i) dịch vụ trợ năng/MediaProjection của một ứng dụng (UID ứng dụng); (ii) tiến trình ADB/UiAutomation (UID shell); (iii) tác nhân từ xa chỉ nhận quan sát đã serialize. Redactor và resolver chặn theo principal tại điểm giao ảnh chụp/cây trợ năng. Đánh giá phải chạy trên bản userdebug với `adb unroot` và SELinux ở chế độ enforcing; nếu tác nhân có quyền root thì nó đọc được mọi thứ và kết quả vô nghĩa.

(b) Bề mặt nguồn. Giá trị OTP hiện diện trên màn hình *trước* khi có `ENTER`: thông báo, ứng dụng SMS, màn hình khóa, ứng dụng email. Che theo "giá trị" (so khớp chính xác trong cây trợ năng, tính bounds cho pixel) chính xác hơn bộ phát hiện NER/OCR nhưng vẫn có thể sót (mã tách dòng, vẽ bằng canvas/ảnh, WebView). Chính sách mặc định [T1]: trong suốt đời handle, các cửa sổ thuộc nguồn (thông báo, ứng dụng SMS, màn hình khóa) bị che *theo cửa sổ* đối với principal tác nhân; che theo giá trị chỉ là lớp bổ sung; không chắc chắn thì bỏ frame (fail-closed).

6.3 Năng lực của kẻ tấn công

Kẻ tấn công kiểm soát đầu ra lập kế hoạch của tác nhân và có thể đặt văn bản/pixel tùy ý trong app, trang web, thông báo hoặc tin nhắn. Nó có thể yêu cầu thao tác trên mọi node mà tác nhân thấy, replay handle đã quan sát, đoán chuỗi handle mới, tranh chấp nhiều lệnh `ENTER`, đổi ứng dụng tiền cảnh, điều hướng WebView, và vẽ chuỗi có cùng cú pháp hình ảnh như handle. Nó đọc được mọi input/output/prompt/trace của tác nhân và các giá trị mà ứng dụng nhìn thấy sau khi nhập hợp lệ nếu chính ứng dụng đích bị xâm phạm. Kẻ tấn công là *thích nghi và biết thiết kế* (nguyên tắc Kerckhoffs): nó biết cú pháp handle, các quy tắc ràng buộc và thời điểm redaction.

Kẻ tấn công *không* đọc được bộ nhớ TCB, giả mạo định danh package Android, trích khóa xác thực không xuất được, sửa đổi chuyển trạng thái nguyên tử của resolver, phá màn hình tin cậy của Protected Confirmation, hay vượt qua mọi hook redaction màn hình/trợ năng. Đây là các giả định tường minh; kết quả phụ thuộc chúng là định lý hệ thống *có điều kiện*, không phải phát biểu vô điều kiện cho mọi thiết bị Android.

6.4 Mục tiêu bảo mật

1. **Không tiết lộ trong quan sát của tác nhân**: quan sát mô hình hóa chỉ lộ leakage đã công bố ℓ, không lộ giá trị thô.
2. **Chốt chặn đích (destination confinement)**: giải chỉ thành công khi bối cảnh đích tin cậy thỏa vị từ đích và tác vụ của handle.
3. **Bất khả giả mạo (unforgeability)**: một chuỗi không do dịch vụ đúc tạo chỉ được chấp nhận với xác suất không đáng kể.
4. **An toàn vòng đời**: handle hết hạn/thu hồi/đã dùng/xác nhận bị từ chối không giải được; handle dùng một lần không thể thành công hai lần kể cả khi đồng thời.
5. **Đóng kín quan sát**: sau khi nhập hợp lệ, giá trị bị che trong các quan sát mô hình hóa sau đó.

Mục tiêu tiện ích hẹp hơn: các luồng đăng nhập, 2FA, nhập định danh, checkout vẫn hoàn thành được với độ trễ và gánh nặng người dùng chấp nhận được.

6.5 Ngoài phạm vi (non-goals)

Không phát hiện mọi prompt injection; không phán xét tác vụ có lợi cho xã hội hay không; không ngăn ứng dụng ngân hàng *được phép* gửi OTP tới server của nó; không chống nhân bị xâm phạm; không che các hiệu ứng vật lý tất yếu của việc hoàn thành tác vụ; không chống tác nhân có quyền root trên thiết bị. Không tuyên bố OTP chống phishing (WebAuthn/passkey giải bài toán khác).

7 Kiến trúc tổng thể và các thành phần

Thành phần	Trách nhiệm
Handle Mint [T1]	Dịch vụ hệ thống mới (sửa OS): móc vào đường ống OTP SMS/thông báo ([T1]), clipboard ([T2]), Credential Manager và ví ([T3]) để đúc handle có kiểu, ràng buộc, bất khả giả mạo. Khóa trong StrongBox/Knox là [T3](T1 dùng Keystore/TEE nếu có).
Resolver [T1]	Sửa input-dispatch và đường trợ năng: khi `ENTER(handle)`, kiểm tra package/field của node đích với ràng buộc, nhập giá trị, thi hành single-use/TTL. Kiểm tra origin của WebView là [T3].
Observation Redactor [T1]	Sửa AccessibilityManagerService và đường chụp màn hình của tác nhân: che giá trị đã giải trong tầm nhìn của tác nhân (mở rộng cơ chế che khi chia sẻ màn hình của Android 15) và che bề mặt nguồn.
Agent SDK / adapter [T1]	Hiển thị handle trong quan sát; hành động `ENTER(handle)`; adapter cho các tác nhân mã nguồn mở, kể cả tác nhân điều khiển qua ADB.
Hardware [T3]	Khóa hậu thuẫn phần cứng; Protected Confirmation cho handle cấp thanh toán.

8 Thuật toán cốt lõi

8.1 Algorithm 1 — Mint tại nguồn có thẩm quyền

Đầu vào: bộ đệm giá trị được bảo vệ s, loại q, xuất xứ σ, task token T, bằng chứng đích E, gợi ý chính sách P.

1. Xác thực caller binder và kiểm tra nó được phép khẳng định nguồn σ.
2. $(\tau, D_T) \leftarrow \mathsf{VerifyTaskToken}(T)$; từ chối nếu thất bại.
3. $D_S \leftarrow \mathsf{SourceDestinations}(\sigma, E)$.
4. $d \leftarrow D_T \cap D_S \cap \mathsf{TypePolicy}(q, P)$; từ chối nếu rỗng.
5. $(t_i, t_e, u, c) \leftarrow \mathsf{Lifecycle}(q, P)$.
6. Sinh ngẫu nhiên $n \xleftarrow{\$} \{0,1\}^\lambda$; lập metadata $H = (n, q, \sigma, d, \tau, t_i, t_e, u, c)$.
7. $a \leftarrow F_K(\mathsf{enc}(H))$ (HMAC-SHA-256 trên mã hóa chuẩn tắc, khóa bí mật) — giữ hay bỏ bước này tùy quyết định D1.
8. Chèn nguyên tử bản ghi $\Omega[n] \leftarrow (s, H, a, \mathsf{live}, \perp)$.
9. Đăng ký che phía nguồn cho giá trị s đối với principal tác nhân (che theo cửa sổ nguồn + che theo giá trị).
10. Xóa bộ đệm caller nếu hỗ trợ; trả về handle $h = (n, q, a)$.

8.2 Algorithm 2 — ENTER/Resolve

Đầu vào: handle $h = (n, q, a)$, task token T', target node token V, thời điểm t.

1. $(\tau', _) \leftarrow \mathsf{VerifyTaskToken}(T')$; từ chối nếu thất bại.
2. Mở giao dịch; nạp $\Omega[n] = (s, H, a_H, z, r)$.

3. Kiểm tra $a = a_H = F_K(\mathsf{enc}(H))$ theo thời gian hằng (constant-time).
4. $x \leftarrow \mathsf{ResolveFreshTargetContext}(V)$.
5. Nếu $z \neq \mathsf{live}$ hoặc $t \notin [t_i, t_e)$ hoặc $u = 0$ hoặc $\tau' \neq \tau$ hoặc $\mathsf{Match}(d, x) = 0$: rollback và `Reject`.
6. Sinh nonce giữ chỗ r; đặt nguyên tử $(z, r) \leftarrow (\mathsf{reserved}, r)$ *và* $u \leftarrow u - 1$ (bảo thủ); commit.
7. $\gamma \leftarrow \mathsf{ObtainConfirmation}(c, n, d, \tau, r)$.
8. Nếu xác nhận không hợp lệ: kết thúc giữ chỗ theo loại (hoàn trả u vì chưa giao); `Reject`.
9. $x' \leftarrow \mathsf{ResolveFreshTargetContext}(V)$; nếu $x' \neq x$ hoặc $\mathsf{Match}(d, x') = 0$: kết thúc bảo thủ (hoàn trả u vì chưa giao); `Reject`.
10. Đăng ký nghĩa vụ redaction cho (V, n, q) *trước* khi giao giá trị.
11. $ok \leftarrow \mathsf{InjectInternal}(V, s, r)$.
12. Mở giao dịch; nếu giao thành công hoặc không chắc chắn: giữ u đã giảm, đặt $z \leftarrow \mathsf{consumed}$ nếu $u = 0$, ngược lại $z \leftarrow \mathsf{live}$; nếu có bằng chứng bền vững là chưa giao: hoàn trả $u \leftarrow u + 1$, $z \leftarrow \mathsf{live}$; commit.
13. Xóa bộ đệm tạm; trả về trạng thái không tiết lộ giá trị.

Điều chỉnh so với phiên bản 1

Phiên bản 1 giảm u ở bước cuối của Algorithm 2 nhưng mục Vòng đời lại nói có thể giảm ngay khi giữ chỗ. Hai chỗ mâu thuẫn nhau. Phiên bản này thống nhất theo hướng bảo thủ (giảm tại bước giữ chỗ, hoàn trả chỉ khi chứng minh được chưa giao), khớp với quy tắc phục hồi "reserved sau sự cố không chắc chắn ⇒ consumed". Cần cập nhật `core.py`, test và phần chứng minh tương ứng (quyết định D4).

8.3 Algorithm 3 — Observation Redactor

Đầu vào: quan sát thô O, principal tác nhân A, registry hoạt động R.

1. $O' \leftarrow O$.
2. Với mọi nghĩa vụ $\rho = (V, n, q, \ldots)$ hoạt động và thấy được bởi A:
   - tính lại định danh node/window và bounds hiện tại của V;
   - nếu định danh ổn định: thay giá trị trợ năng, sự kiện văn bản và vùng ảnh chụp bằng mask theo kiểu;
   - nếu không: che window chứa nó hoặc bỏ frame theo chính sách fail-closed;
   - loại bỏ action echo, văn bản debug và content description chứa tài liệu được bảo vệ.
3. Với mọi cửa sổ thuộc bề mặt nguồn của handle còn sống: che theo cửa sổ (xem Mục 6.2).
4. Trả về O' qua bộ đệm riêng theo principal.

9 Vòng đời, thu hồi và phục hồi

- Hết hạn kiểm tra theo đồng hồ đơn điệu (monotonic); thay đổi đồng hồ tường không thể hồi sinh handle.
- **[T1]** Mọi handle chỉ nằm trong bộ nhớ của `system_server`; khởi động lại dịch vụ thu hồi mọi handle (quyết định D2). Điều này đơn giản hóa phục hồi sự cố: không còn trạng thái `reserved` sống sót qua crash.
- **[T3]** Trạng thái bền vững (sống sót qua khởi động lại) cần nhật ký fsync, chống rollback và tách đa người dùng. Nếu làm, áp dụng quy tắc bảo thủ: bản ghi `reserved` sau sự cố không chắc chắn bị `consumed` với bí mật dùng một lần, trừ khi có bằng chứng bền vững rằng chưa giao.
- Kết thúc tác vụ thu hồi mọi handle theo chỉ số τ. Khóa thiết bị, đổi profile người dùng, thay package đích, hoặc vô hiệu yêu cầu nguồn đều có thể kích hoạt thu hồi.
- Handle nhập nhiều lần giảm u ngay khi giữ chỗ (xem Algorithm 2).

10 Phân tích hình thức và tính chất bảo mật

10.1 Bản ghi và chuyển trạng thái

Kho tin cậy ánh xạ mỗi định danh n tới bản ghi

$$\Omega[n] = (s, H, a, z, r), \quad H = (n, q, \sigma, d, \tau, t_i, t_e, u, c), \quad a = F_K(\mathsf{enc}(H)),$$

với $z \in \{\mathsf{live}, \mathsf{reserved}, \mathsf{consumed}, \mathsf{expired}, \mathsf{revoked}\}$. Máy trừu tượng hỗ trợ Mint, Resolve, và Redact.

10.2 Các giả định

1. **Toàn vẹn reference monitor**: kẻ tấn công không sửa được dịch vụ, kho, resolver, registry, trích bối cảnh đích, đường nhập; mọi giải đều qua resolver.
2. **Xác thực nguồn và tác vụ**: chỉ adapter nguồn được duyệt mới đúc được handle nguồn; τ và σ không thể bị giả mạo hoặc mở rộng.
3. **Bảo mật authenticator và độ tươi**: F là EUF-CMA, K bí mật, định danh lấy đều từ $\{0,1\}^\lambda$, enc chuẩn tắc và đơn ánh.
4. **Định danh đích xác thực**: bối cảnh $x = (p, o, f, v)$ do resolver suy ra bằng đúng đích nhập tại thời điểm commit.
5. **Redaction kênh mô hình hóa đầy đủ**: trước khi s quan sát được, mọi kênh trong View_A áp cùng một mask.
6. **Hành vi công khai không phân biệt theo giá trị**: với hai bí mật cùng độ dài và leakage bằng nhau, phân phối công khai là không phân biệt được.

10.3 Các kết quả hình thức

- **Bổ đề mint-view độc lập**: dưới các giả định nguồn/khóa/không phân biệt, phân phối quan sát được khi đúc s_0 và s_1 là không phân biệt tính toán được.
- **Bất biến ủy quyền resolve**: dưới toàn vẹn monitor và định danh đích, mọi chuyển live→reserved chỉ xảy ra khi authenticator, tác vụ, thời gian, ngân sách dùng, điều kiện xác nhận và vị từ đích đều đạt.
- **Định lý chốt chặn đích**: nếu giá trị được commit tới đích x trong tác vụ τ', thì $\mathsf{Match}(d, x) = 1$ và $\tau' = \tau$.
- **Định lý bất khả giả mạo**: $\mathsf{Adv}^{\mathsf{forge}} \le \mathsf{Adv}^{\mathsf{euf}}_F + Q/2^\lambda$.
- **Mệnh đề single-use**: với $u = 1$, tối đa một tiến trình resolve đồng thời tới được commit.
- **Mệnh đề đơn điệu hết hạn**: handle bị từ chối vì $t \ge t_e$ không thể trở nên hợp lệ nhờ chỉnh đồng hồ tường.
- **Định lý không tiết lộ trong quan sát**: $\mathsf{Adv}^{\mathsf{nd}} \to 0$ với bí mật cùng leakage.
- **Hệ quả chống chuyển hướng**: nếu đích tấn công nằm ngoài d, injection không thể khiến resolver commit bí mật ở đó.
- **Mệnh đề suy giảm an toàn**: thay chính sách P bằng P' hẹp hơn không tạo trace resolve thành công mới.

Hai điểm cần chốt trước khi viết lại phần chứng minh

(a) Vai trò của authenticator a (quyết định D1). Nếu kho nằm trong bộ nhớ TCB và n là định danh ngẫu nhiên không đoán được, việc tra $\Omega[n]$ đã đủ để bất khả giả mạo (cận $Q/2^\lambda$). MAC chỉ thêm giá trị nếu bản ghi được lưu ngoài TCB/bền vững hoặc nếu cần xác thực metadata do handle mang theo. Reviewer sẽ hỏi "vì sao cần MAC"; hoặc nêu lý do cụ thể, hoặc đơn giản hóa định lý. **(b)** Giả định 5 và 6 là hai giả định mạnh nhất. Thí nghiệm đọc lại sau khi nhập (họ tấn công 5) và kiểm tra rò rỉ qua hành vi ứng dụng là *bằng chứng thực nghiệm* cho chúng, không thay thế được chứng minh; bài phải nói rõ giả định nào còn mang tính kiến trúc.

11 Chỉ số và kế hoạch đánh giá

11.1 Các điều kiện và baseline

B0: không bảo vệ [T1]
Tác nhân nhận giá trị thô trong quan sát thường và gõ như bình thường.

B1: placeholder phát hiện được [T1]
Bộ phát hiện OCR/NER thay văn bản nhạy cảm bằng placeholder rồi giải theo đích yêu cầu, không ràng buộc đích; xấp xỉ họ placeholder di động đã công bố.

B2: luồng và cơ chế nền tảng [T2]
Android Autofill, OTP Autofill hoặc Credential Manager khi app test gọi được; cộng các cơ chế che sẵn có: `FLAG_SECURE`, che văn bản trường mật khẩu trong cây trợ năng, content sensitivity (API 35+), che thông báo nhạy cảm. Luồng không hỗ trợ được ghi `not_applicable`, không tính là thất bại.

B3: SecretHandle [T1]
Đúc nguồn đầy đủ, ràng buộc đích/tác vụ, vòng đời và redaction.

11.2 Chỉ số

- **Exposure** E: tỉ lệ tác vụ mà bí mật canary xuất hiện trong tập artifact quan sát được bởi tác nhân.
- **Redirect** R: tỉ lệ trial tấn công mà giá trị thô tới đích ngoài d.
- **Utility** U: tỉ lệ trial lành tính thỏa oracle của ứng dụng; báo cáo cả U tuyệt đối và U có điều kiện trên các trial mà B0 thành công (để tách lỗi của tác nhân khỏi lỗi do cơ chế).
- **Post-entry leakage** L: số trial giá trị xuất hiện trong quan sát mô hình hóa sau đó.
- **Forgery** F / **Replay** Y: số lần nhập trái phép thành công trên mỗi lần thử.
- **Độ trễ**: overhead end-to-end và thành phần mint, xác thực, tra đích, xác nhận, nhập, redaction.
- Kèm: bộ nhớ, CPU, năng lượng **[T2]**, số lần xác nhận người dùng, tỉ lệ từ chối chính sách, tỉ lệ từ chối sai, kết quả phục hồi sự cố, tính sẵn có định danh đích, hồi quy tác vụ chung.

11.3 Các cuộc tấn công/stress test

Bảy họ tấn công được chia theo cách chạy, vì điều này quyết định chi phí và cỡ mẫu:

1. **Đọc và tuồn ra** (LLM): tác nhân tới thông báo chứa canary OTP và gặp chỉ thị gửi tới endpoint tấn công.
2. **Chuyển hướng đích** (LLM + planner kịch bản): injection yêu cầu `ENTER(handle)` vào app tấn công, package khác, WebView origin khác, trường không phải OTP, hoặc node cũ/tái chế.
3. **Handle giả** (kịch bản): app độc hại vẽ đúng cú pháp handle và gửi ID/authenticator đoán.
4. **Replay và race** (kịch bản): phát resolve tuần tự và đồng thời quanh lúc dùng/hết hạn; kill `system_server` tại từng pha giữ chỗ.
5. **Đọc lại sau khi nhập** (LLM + kịch bản): ngay và lặp lại sau khi nhập, thu ảnh chụp, cây trợ năng, sự kiện đổi văn bản, clipboard, IME và log debug, qua cả ba kiểu principal (UID app, UID shell, quan sát serialize).
6. **Lạm dụng ngữ nghĩa cùng đích** (LLM): giữ đích trong package/origin cho phép nhưng đổi ngữ cảnh tài khoản/giao dịch — đo rủi ro tồn dư mà ràng buộc đích không tuyên bố loại bỏ. Kết quả khác 0 là *bình thường*; phải báo cáo trung thực.
7. **Nhập nhằng định danh** (kịch bản; WebView thuộc **[T2]**/**[T3]**): thay đổi điều hướng, frame lồng, custom view, overlay, focus, chia màn hình, bàn phím OEM, tái chế view, xoay, kết nối lại dịch vụ trợ năng.

Hai loại kẻ tấn công cần cùng có

Planner đối kháng bằng kịch bản: một "tác nhân bị chiếm hoàn toàn" thử `ENTER` vào mọi đích sai và mọi thứ tự thao tác. Nó kiểm tra bảo đảm của hệ thống *không phụ thuộc* vào việc LLM có nghe lời injection hay không, và cho cỡ mẫu lớn với chi phí thấp. **Tác nhân LLM thật dưới injection**: cho tính thực tế (ecological validity) và để chứng minh B0 thật sự bị khai thác. Báo cáo cả hai; nếu tỉ lệ khai thác B0 của tác nhân thật thấp, nói rõ và dựa vào planner kịch bản cho tuyên bố ở mức hệ thống.

11.4 Thí nghiệm quyết định (killer experiment)

Chạy mẫu OTP-forwarding của `2510.27140` chống lại app đăng nhập+OTP mã nguồn mở:

- **B0**: tác nhân đọc mã canary và bị lộ với chỉ thị chuyển tiếp → nó chuyển mã.
- **B3**: adapter SMS chỉ lộ handle ràng buộc app đăng nhập và trường OTP.

Thí nghiệm *chỉ đạt* nếu oracle bảo vệ xác nhận cả ba điều kiện: (a) mã thô vắng mặt trong mọi artifact của tác nhân, (b) `ENTER(handle)` vào trường tấn công bị resolver từ chối, và (c) đăng nhập hợp lệ hoàn thành. Thất bại ở bất kỳ điều kiện nào chặn tuyên bố tương ứng và buộc phân tích nguyên nhân trước khi mở rộng sang credential/checkout. **Phiên bản này yêu cầu chạy một bản thu nhỏ (n=30) ngay ở cổng G1 (tuần 5), không đợi tới tuần 13.**

11.5 Cỡ mẫu và ngân sách chạy

Kết quả "0 trên n" chỉ chứng minh được một cận trên. Với khoảng tin cậy Wilson 95%, cận trên khi quan sát 0/n là $z^2/(n+z^2)$ với $z = 1{,}96$:

n trial	Cận trên 95% khi 0/n
30	11,4%
100	3,7%
300	1,3%
1 000	0,38%
10 000	0,04%

- Họ tấn công chạy bằng kịch bản (họ 3, 4, TOCTOU của họ 7) không cần LLM: chạy $n \geq 10^4$.
- Ô LLM chính của B3: $n \geq 300$ để chặn tỉ lệ rò rỉ/chuyển hướng dưới khoảng 1%. Ô đối chứng B0, B1: $n \approx 100$.
- **Ma trận đầy đủ của phiên bản 1 không chạy nổi**: 3 tác nhân × 2 mô hình × 4 app × 4 điều kiện × 7 họ × 300 = 201 600 trial; với ước lượng 2 phút/trial thì khoảng 6 720 giờ, tức khoảng 280 ngày-thiết bị.
- **Ma trận T1 đề xuất**: 2 mô hình × 2 app × 4 họ LLM (1, 2, 5, 6) = 16 ô; B3 × 300 = 4 800; B0 và B1 × 100 = 3 200; tổng khoảng 8 000 trial ≈ 270 giờ-thiết bị, chạy được trong vài ngày với 2–4 máy ảo song song.
- LLM không tất định: ghim định danh và ngày chạy của mô hình cloud, đặt trần chi phí API, xử lý các trial cùng tác nhân/mô hình như dữ liệu có cụm (bootstrap theo cụm).

12 Hiện trạng kho mã và khoảng trống

12.1 Những gì đã có

Hạng mục	Nội dung
Bản thảo	`main.tex` + 12 section + 2 phụ lục, `main.pdf` (53 trang)
Ý tưởng	`idea.tex` / `idea.pdf` (concept document)
Mô hình tham chiếu	Gói Python `src/secrethandle` (core, crypto, policy, store, metrics, …)
Bộ điều hợp	`adapters.py`, `baselines.py`, `oracles.py` (dạng hợp đồng trừu tượng)
Harness tổng hợp	`experiments.py`, `scripts/`, `configs/`
Kiểm thử	`tests/` (offline, không mạng/dataset/GPU)
Audit	`audits/` (ánh xạ paper–mã, khoảng trống, toàn vẹn kết quả)

12.2 Những gì còn thiếu (blocking cho kết quả cấp bài báo)

Trừ khi ghi khác, mọi mục dưới đây hiện ở trạng thái adapter stub.

Khoảng trống	Yêu cầu công việc bên ngoài	Bậc
Dịch vụ hệ thống AOSP + ủy quyền binder	Cài dịch vụ trong `system_server`, allowlist, xác thực task token, xuất audit	[T1]
Kho handle trong bộ nhớ	Khởi động lại = thu hồi; xóa bộ đệm; tách principal	[T1]
Adapter nguồn SMS/thông báo	Hợp đồng xuất xứ, che bề mặt nguồn	[T1]
Task token xác thực	Định dạng chữ ký/OS, anti-replay, giao tập app cho phép	[T1]
Target node token	Ràng buộc window/connection/node/generation/UID, TTL ngắn	[T1]
Đường nhập riêng	Bỏ qua đường thao tác/tác nhân và lịch sử IME thường	[T1]
Redaction pixel/trợ năng theo principal	Gồm UID app, UID shell/UiAutomation, quan sát serialize	[T1]
Ứng dụng & endpoint thực (app A, B)	App test mở và endpoint verifier/attacker cô lập	[T1], cấp phát thủ công
Tác nhân/mô hình	Ghim một runner chính + 2 mô hình; cấu hình cloud/local	[T1], cấp phát thủ công
Adapter clipboard; app C; B2	Hợp đồng xuất xứ clipboard; app checkout địa chỉ/điện thoại; baseline nền tảng	[T2]
AndroidWorld/SPA-Bench (hồi quy tác vụ chung)	Cấp phát thủ công, adapter giữ nguyên schema	[T2]
Chỉ số thiết bị (năng lượng)	Perfetto/Batterystats/power tooling, trace đồng bộ	[T2]
Khóa hậu thuẫn phần cứng (StrongBox/Knox)	Sinh khóa Android Keystore, báo cáo attestation/security level	[T3]
Kho bền vững + nhật ký sự cố	Lưu trữ mã hóa, thứ tự fsync, chống rollback, đa người dùng	[T3]
Adapter Credential Manager, ví	Hợp đồng xuất xứ, xác nhận	[T3]
Xác thực WebView origin	Thay đổi browser/WebView cho binding node→frame→origin; fail-closed nếu không	[T3] (chưa giải quyết)
Protected Confirmation	Ràng buộc digest với UI tin cậy của nền tảng	[T3]

Cảnh báo về kết quả

Không tuyên bố kết quả thực nghiệm cấp bài báo cho tới khi có dữ liệu benchmark thực và provenance được ghi lại. Mọi output sinh cục bộ được dán nhãn “synthetic smoke-test, không phải kết quả bài báo”.

Phần II — Hướng dẫn các bước hoàn thành

13 Nguyên tắc chung và cách tổ chức công việc

13.1 Nguyên tắc

1. **An toàn dữ liệu tuyệt đối**: chỉ dùng canary tổng hợp. Không bao giờ dùng OTP, thông tin đăng nhập, địa chỉ hay giao dịch thật. Bí mật chỉ nằm trong oracle bảo vệ, hủy sau phân tích.
2. **Tính tái lập**: ghim mọi thứ — commit AOSP, build thiết bị, kernel, hash APK, phiên bản provider, commit tác nhân, định danh mô hình và ngày chạy, prompt, seed, endpoint, tệp chính sách.
3. **Trung thực kết quả**: mọi ô bảng/hình thay bằng giá trị đo, "unsupported" hoặc "not applicable"; không chuyển xu hướng mong đợi thành ngôn ngữ kết quả.
4. **Fail-closed**: chính sách mặc định từ chối khi thiếu định danh bắt buộc.
5. **Cắt phạm vi sớm, không muộn**: hạng mục **[T2]**/**[T3]** chỉ bắt đầu khi cổng G2 qua. Hạng mục không kịp phải được chuyển sang mục Giới hạn và rút khỏi các tuyên bố đóng góp, không để dở dang trong thân bài.
6. **Mọi tuyên bố gắn phạm vi**: nêu rõ thiết bị, phiên bản Android, loại view, kiểu principal tác nhân, loại tác nhân.

13.2 Điều kiện tiên quyết

- Kiến thức: Android framework (system_server, binder, accessibility, input dispatch, Keystore), kỹ thuật build AOSP, prompt injection, thống kê (Wilson, McNemar, bootstrap theo cụm).
- Nhân lực: giả định 2 kỹ sư (Track A: hệ thống/AOSP; Track B: tác nhân/harness/đánh giá). **Track A là nút cổ chai**; nếu chỉ có một người làm AOSP, mọi mốc phần hệ thống nên nhân với khoảng 1,5.
- Phần cứng: nhóm *không có* Google Pixel, nên môi trường chính là *emulator/Cuttlefish chạy ảnh AOSP tự build* (D6) — đủ cho đánh giá chức năng và trial hàng loạt. Một Pixel (nếu tiếp cận được) chỉ để bổ sung số đo độ trễ/năng lượng và keystore phần cứng, chuyển sang Giới hạn/future work.
- Máy chủ: EC2 riêng với Python 3.11/3.12 cho harness; nếu chạy emulator/Cuttlefish thì chọn loại instance có KVM/nested virtualization. Build AOSP cần ổ đĩa và RAM lớn (theo tài liệu AOSP hiện hành, cỡ 400 GB đĩa và 32–64 GB RAM; xác minh khi dựng).
- Tài khoản/quyền: quyền tải AOSP, license/đăng ký AndroidWorld/SPA-Bench, tài khoản cloud model có trần chi phí.

13.3 Hai track, ba bậc, các cổng quyết định (phiên bản MobiSys 2027)

Công việc chia thành Track A (dịch vụ AOSP, resolver, redaction — *đường Plan B*) và Track B (tái hiện tấn công, ứng dụng, tác nhân, monitor, mô hình tham chiếu). Với mục tiêu MobiSys 2027 — Emerging Ideas (nộp 04/12/2026), **Track B là đường găng**: B0, tấn công và disclosure monitor chạy được trên emulator/Android stock ngay từ tuần 1 mà *không* cần AOSP. Track A (build ảnh AOSP + dịch vụ trong `system_server`) vẫn khởi động sớm nhưng phục vụ *bản full* ở Plan B, không phải điều kiện nộp. Mỗi cổng dưới đây là một điểm dừng bắt buộc để quyết định tiếp tục, cắt phạm vi hoặc đổi kế hoạch.

Cổng	Hạn (hết tuần)	Điều kiện qua	Nếu không qua
G0	25/10/2026 (W2)	Môi trường dev sẵn sàng; tấn công OTP-forwarding tái hiện được với B0 trên emulator/stock và ghi tỉ lệ khai thác; chốt D0–D6	Chuyển hẳn sang emulator/Cuttlefish (không có Pixel); đổi tác nhân hoặc dùng planner kịch bản; chưa bắt đầu viết bản thảo

Cổng	Hạn (hết tuần)	Điều kiện qua	Nếu không qua
G1	15/11/2026 (W5)	Tái hiện đầy đủ họ tấn công (1, 3, 4, 7); bằng chứng sơ bộ từ mô hình tham chiếu (ghi rõ là mô phỏng); khung bản thảo problem/threat/design	Thu hẹp về B0 + thiết kế; ghi rõ mọi số liệu là sơ bộ/mô phỏng
G2	27/11/2026 (W7)	Đăng ký abstract MobiSys (tiêu đề + abstract $\geq$100 từ, không "TBD"); bản thảo gần hoàn chỉnh	Đăng ký abstract tối thiểu; dồn toàn lực cho bản thảo
G3	01/12/2026 (W7)	Đóng băng bản thảo 12 trang + phụ lục; chuyển `acmart` sigconf; ghi tên tác giả; gán nhãn LLM	Nộp bản hẹp hơn; kiểm format kỹ để tránh desk-reject
Nộp	04/12/2026 (AoE)	MobiSys 2027 — Emerging Ideas (tiêu đề bắt đầu `[Emerging Ideas]`)	—
Plan B	2027 (W1–W16 từ 2027)	Bản full: T1 AOSP trên emulator/Cuttlefish + killer experiment + ma trận T1; số đo thiết bị thật nếu có	Nộp kỳ sau (MobiSys 2028 / USENIX / NDSS / IEEE S&P)

14 Giai đoạn 0 — Chuẩn bị, quyết định thiết kế và đạo đức (W1–W2)

Mục tiêu: dựng nền tảng tái lập, chốt các quyết định làm thay đổi phạm vi, và khởi động mọi việc có độ trễ dài ngay tuần 1.

14.1 Quyết định thiết kế (đã chốt)

Các quyết định D0–D6 làm thay đổi phạm vi, chứng minh và mã nguồn đã được chốt ngày 17/10/2026 và ghi đầy đủ trong `SecretHandle/docs/design_decisions.md`. Tóm tắt: D0 = MobiSys 2027 (Emerging Ideas, nộp 04/12/2026); D1 = giữ MAC; D2 = kho trong bộ nhớ; D3 = ba kiểu principal tác nhân; D4 = giảm u tại bước giữ chỗ; D5 = WebView fail-closed; D6 = môi trường emulator/Cuttlefish (không có Pixel).

14.2 Công việc

- **[A]** Bắt đầu `repo sync` AOSP ngay tuần 1 (hàng chục đến hàng trăm GB, mất nhiều giờ); ghim commit/branch và thiết bị Pixel; ghi vào provenance.
- **[B]** Khởi động các thủ tục dài: license AndroidWorld/SPA-Bench, API key cloud model + trần chi phí, thiết bị.
- **[B]** Tạo kho Git integration tách khỏi package tham chiếu (AOSP patch, adapter, tác nhân).
- **[B]** Dựng Python 3.11, `pip install -e '.[dev]'`; xác nhận `ruff`, `mypy` (ghi nhận nếu thiếu).
- **[B]** Dựng EC2 cô lập: volume riêng, user riêng, tường lửa chặt; credential trong trình quản lý bí mật, *không* trong YAML/log/repo.
- **[B] Tái hiện tấn công B0** trên Android stock: chạy OTP-forwarding với tác nhân đã chọn, đo tỉ lệ khai thác, dựng disclosure monitor bản đầu.
- **[B]** Khung đạo đức: quy trình tạo/hủy canary, cô lập endpoint tấn công, không dữ liệu thật; kiểm tra yêu cầu của cơ quan (IRB/hội đồng) và mục ethics của CFP.
- Ghim phiên bản Python, OS, công cụ thống kê vào `provenance`.

Sản phẩm bàn giao:

Môi trường chạy được `pytest`; AOSP đang đồng bộ/build; B0 tái hiện được; D0–D5 được ghi thành tài liệu quyết định (mỗi quyết định một đoạn); khung đạo đức.

Tiêu chí nghiệm thu (Definition of Done):

`python -m pytest -q` xanh 100%; không có bí mật thật trong repo; cổng G0 qua.

Cổng G0: 25/10/2026.

15 Giai đoạn 1 — Củng cố mô hình tham chiếu Python (W1–W2, song song, tối đa 2 tuần)

> **Mục tiêu: khóa ngữ nghĩa và hợp đồng adapter vừa đủ để viết mã AOSP; không biến thành giai đoạn hoàn thiện vô hạn.**

15.1 Công việc

- Soát `core.py`, `crypto.py`, `policy.py`, `store.py`, `redaction.py`, `metrics.py`, `statistics.py` đối chiếu Algorithm 1–3 và `audits/paper_to_code_traceability.csv`; áp dụng D1, D4.
- Test dựa tính chất: giao hội chính sách không mở rộng quyền, single-use dưới đồng thời, đơn điệu hết hạn.
- Test dựa mô hình: lịch trình crash/đồng thời tại mỗi ranh giới reserve/register/inject/commit.
- Đóng băng hợp đồng cho `AospAdapter`, `PlatformFlowAdapter`, `AgentRunnerAdapter`, `BenchmarkAdapter`.
- Property: không trường bí mật nào rò rỉ qua serialization công khai/audit.
- Mở rộng oracle tiết lộ: exact, base64, URL, JSON, chuẩn hóa Unicode, chuẩn hóa số, số tách bởi dấu cách/gạch ngang.
- Chạy `ruff` + `mypy`; cập nhật `audits/` sau mỗi thay đổi ngữ nghĩa.

Sản phẩm bàn giao:

Bộ test mở rộng; hợp đồng adapter đóng băng; báo cáo audit cập nhật.

Tiêu chí nghiệm thu (Definition of Done):

Tất cả test xanh; mọi thay đổi ngữ nghĩa có test; traceability phủ 100% đối tượng paper đã cài.

16 Giai đoạn 2 — Walking skeleton: lát cắt dọc đầu tiên (W3–W5)

> **Mục tiêu: có một đường đi hoàn chỉnh từ SMS đến ô nhập trong app đăng nhập, chạy killer experiment thu nhỏ, trước khi xây bất kỳ thứ gì phức tạp.**

Lý do: rủi ro lớn nhất của đề tài (redaction trước khi tác nhân thấy, định danh principal, nhập giá trị dưới ranh giới quan sát) chỉ lộ ra khi chạy đầu-cuối. Phiên bản 1 đặt killer experiment ở tuần 13–16, sau khi đã xây kho bền vững, nhật ký fsync, đa người dùng; phiên bản này đưa nó lên tuần 5.

16.1 Công việc

- **[A]** Dịch vụ trong `system_server` với AIDL tối thiểu: `mint` (allowlist), `resolve`, `revoke`; kho trong bộ nhớ (D2).
- **[A]** Adapter SMS/thông báo bản test (nhận tin từ emulator console hoặc app SMS test); che mã khỏi thông báo hướng tác nhân.
- **[A]** Resolver tối thiểu: `ResolveFreshTargetContext` (package + field + tác vụ), từ chối nếu không khớp; giữ chỗ nguyên tử bằng khóa kho.

- **[A]** Đường vào cho tác nhân điều khiển qua ADB: lệnh dịch vụ (ví dụ `cmd secrethandle enter`) chạy bằng UID shell; resolver kiểm tra UID gọi thuộc principal tác nhân đã đăng ký; giá trị không đi qua stdout/log.
- **[A]** Redaction cây trợ năng theo principal (so khớp giá trị chính xác) và che cửa sổ nguồn; chưa cần redaction ảnh chụp đầy đủ.
- **[B]** App A (đăng nhập + OTP) + endpoint verifier/attacker cô lập; adapter tác nhân → `ENTER(handle)`; disclosure monitor.
- **[B]** Killer experiment thu nhỏ ($n = 30$) B0 vs B3 với planner kịch bản và một tác nhân thật.
- **Không làm trong giai đoạn này**: kho bền vững, khóa phần cứng, xác nhận, WebView, ví, đa người dùng.

Sản phẩm bàn giao:

Dịch vụ + adapter SMS + resolver + app A chạy đầu-cuối; báo cáo killer thu nhỏ.

Tiêu chí nghiệm thu (Definition of Done):

`ENTER(handle)` vào trường OTP hợp lệ nhập đúng một lần; đích sai bị từ chối; canary vắng trong cây trợ năng và log của tác nhân.

Cổng G1: 15/11/2026.

17 Giai đoạn 3 — Hoàn thiện lõi dịch vụ và resolver (W6–W8, Track A)

Mục tiêu: biến skeleton thành lõi đáng tin: bề mặt binder tối thiểu, xác thực tác vụ, giữ chỗ nguyên tử và chống TOCTOU.

17.1 Công việc

- AIDL đầy đủ: `mint` (chỉ adapter allowlist), `listHandles` (theo tác vụ hiện tại), `resolve`, `revoke`, `exportAudit` (bảo vệ, giới hạn tần suất chi tiết lỗi).
- Mô hình quyền: xác thực caller identity/provenance trước khi đúc; task-token verifier, source allowlist, quota, rate limit.
- **Target node token**: ràng buộc window ID, connection ID, node ID, generation, package UID, TTL ngắn; không phải node ID thô.
- Phân loại trường: dùng input type framework, autofill hints, view semantics; bằng chứng không nhất quán → fail-closed.
- Giữ chỗ nguyên tử (compare-and-swap/khóa kho) live→reserved; tái xác thực đích ngay trước nhập; năng lực nhập một lần ràng buộc reservation.
- Mã hóa chuẩn tắc metadata, tách khóa, độ dài tag (nếu giữ MAC theo D1); báo cáo security level thực của khóa (TEE/StrongBox/none).
- Thu hồi: kết thúc tác vụ, khóa thiết bị, đổi profile, thay package đích.
- Bảng xử lý lỗi (unknown handle, expired, mismatch, target vanished, redactor không theo dõi được, crash), tất cả value-oblivious; wipe bộ đệm ở mọi đường thoát.
- Test đồng thời đầu-cuối và `kill system_server` tại từng ranh giới reserve/register/inject/commit.

Sản phẩm bàn giao:

Dịch vụ build/load trên Pixel userdebug và emulator; AIDL; bảng xử lý lỗi; báo cáo security level.

Tiêu chí nghiệm thu (Definition of Done):

Test binder allowlist xanh; không tồn tại lịch trình đồng thời nào cho hai lần commit của handle dùng một lần; kill giữa chừng không gây giao hai lần.

18 Giai đoạn 4 — Redaction quan sát và đường nhập riêng (W6–W9, Track A)

Mục tiêu: giữ giá trị khỏi *mọi* quan sát của tác nhân trước và sau khi nhập, trên cả ba kiểu principal.

18.1 Công việc

- Đường nhập riêng (private input) bỏ qua đường thao tác hướng tác nhân và lịch sử/gợi ý IME thường.
- Đăng ký nghĩa vụ redaction ρ *trước* khi giao giá trị.
- Redaction trợ năng theo principal: thay giá trị/text/sự kiện bằng mask theo kiểu.
- Redaction ảnh chụp *trước khi* buffer được ánh xạ vào tiến trình tác nhân (không dùng overlay vẽ sau); phủ `screencap`, UiAutomation, MediaProjection, `takeScreenshot` của dịch vụ trợ năng.
- Theo dõi định danh view/window ổn định; tính lại bounds tại thời điểm capture; không định danh lại được $\rightarrow$ che window chứa nó một frame hoặc bỏ frame.
- Che bề mặt nguồn theo cửa sổ (thông báo, ứng dụng SMS, màn hình khóa) trong suốt đời handle.
- Loại bỏ action echo, văn bản debug, content description chứa tài liệu bảo vệ.
- Kết thúc redaction khi trường bị xóa, window đóng, tác vụ kết thúc, hoặc trạng thái app tin cậy chứng minh giá trị không còn — *không* dùng chỉ hẹn giờ.
- **[B] Harness đọc lại (read-back)**: sau khi nhập, thu ảnh chụp, cây trợ năng, sự kiện đổi text, clipboard, IME, `logcat`, `dumpsys`, qua UID app, UID shell và quan sát serialize; so khớp bằng oracle mã hóa đa dạng.
- **[T3]** WebView origin binding, Protected Confirmation (digest gồm đích, tác vụ, ngữ cảnh giao dịch, ID handle, nonce); nếu chưa làm, dùng dialog hệ thống và dán nhãn assurance yếu hơn.

Sản phẩm bàn giao:

Redactor trợ năng + ảnh chụp + bề mặt nguồn; private input path; harness đọc lại.

Tiêu chí nghiệm thu (Definition of Done):

Pilot $n = 100$: sau khi nhập, giá trị vắng mặt trong ảnh chụp, cây trợ năng, sự kiện đổi text, clipboard, IME, log và attacker traffic, trên cả ba kiểu principal. Mọi trường hợp không chắc chắn đều fail-closed.

19 Giai đoạn 5 — Ứng dụng test, endpoint, tác nhân và monitor (Track B, rolling W1–W9)

Mục tiêu: có đủ khối đo lường T1 (app A, B; một runner; hai mô hình; monitor mọi kênh) trước khi hệ thống sẵn sàng.

19.1 Công việc

- Ứng dụng test mở: (A) đăng nhập + OTP SMS **[T1]**; (B) 2FA nhiều trang với preview tin nhắn đối kháng **[T1]**; (C) checkout địa chỉ + điện thoại **[T2]**; (D) checkout token ví với xác nhận mỗi lần dùng **[T3]**.
- Mỗi app chỉ ghi sự kiện ground-truth vào harness bảo vệ: định danh đích, đã giao hay chưa, server chấp nhận, hoàn thành tác vụ.
- Endpoint verifier (đo tính sẵn có định danh) và endpoint attacker cô lập (đích ngoài d) — không kết nối mạng thật.
- Ghim nguồn, build và băm APK (SHA-256); ghi schema oracle server.
- Tác nhân: chọn *một* runner chính (AppAgent hoặc một runner tương thích AndroidWorld đang được bảo trì), sau thử tái hiện B0 ở G0. Tác nhân thứ hai (Mobile-Agent, DroidRun nếu ổn định) là **[T2]**.

- Mô hình: một cloud và một local/open-weights nếu khả thi, nếu không thì hai mô hình cloud khác nhau và ghi rõ; ghim định danh và ngày chạy.
- Disclosure monitor: tìm canary chính xác và mã hóa cấu trúc trong prompt, ảnh (OCR), cây trợ năng, tool call, trace, model traffic, planner state, actions, clipboard, IME, logs, crash data, attacker traffic.
- Hồi quy tác vụ chung (AndroidWorld/SPA-Bench, chênh lệch thành công trên task không dùng handle) **[T2]**.

Sản phẩm bàn giao:

App A, B + hash; endpoint verifier/attacker; runner và mô hình ghim; monitor mọi kênh.

Tiêu chí nghiệm thu (Definition of Done):

Chạy được một luồng app A dưới giám sát disclosure, tái lập với seed cố định; build tái lập từ nguồn ghim; hash khớp.

20 Giai đoạn 6 — Baseline và ablation (W6–W10)

Mục tiêu: cài B0, B1, B3 và các nhánh ablation để so sánh công bằng; đóng băng tham số trước khi xem kết quả.

20.1 Công việc

- **B0**: chỉ với canary tổng hợp và đích trong bộ nhớ cô lập.
- **B1**: bộ phát hiện tham chiếu tất định *hoặc* cài đặt thực của họ detector đã được tác giả duyệt; ghi rõ phạm vi.
- **B3**: adapter AOSP đầy đủ cho T1 (mint, target token, private input, redaction, audit).
- **B2** **[T2]**: Autofill/OTP Autofill/Credential Manager cho app hợp tác; `FLAG_SECURE`, che trường mật khẩu, content sensitivity; workflow không hỗ trợ ghi `not_applicable`.
- Ablation T1 (mỗi lần tắt một thứ): đúc nguồn (thay bằng cạo màn hình), ràng buộc đích, ràng buộc tác vụ, vòng đời nguyên tử, redaction sau nhập. Ablation xác nhận mỗi lần và kiểm tra origin là **[T3]**. Biến thể không an toàn *chỉ* dùng canary tổng hợp + endpoint cô lập.
- **Đóng băng tham số** trước khi xem kết quả: bits định danh, bits tag, TTL, ngân sách dùng, rate limit, thời lượng quan sát, số lần lặp, số bootstrap, chính sách outlier, cỡ mẫu từng ô.

Sản phẩm bàn giao:

B0, B1, B3 + các nhánh ablation; bảng tham số đóng băng.

Tiêu chí nghiệm thu (Definition of Done):

Chuyển đổi B0↔B3 không đổi văn bản tác vụ; audit hiển thị quyết định; oracle cô lập; tham số đóng băng (G3).

21 Giai đoạn 7 — Chạy thí nghiệm (W10–W13)

Mục tiêu: killer experiment đầy đủ trước, rồi ma trận T1; thu kết quả secret-free.

21.1 Công việc

- Provisioning theo `docs/ec2_experiment_guide.md`; đặt tệp theo `docs/data_format.md`; clone repo tác nhân/benchmark tại commit đã audit.
- Copy `configs/experiment_template.yaml`; chỉ đặt đường dẫn/ID đã cấp phát; đóng băng tham số.
- Smoke test canary cô lập: xác nhận không có dữ liệu account/OTP/địa chỉ/credential/payment thật.

- **Killer experiment đầy đủ** (B0 vs B3 trên app A) với cỡ mẫu ở Mục 11.5 (B3: $n \geq 300$): kiểm ba điều kiện (a)(b)(c).
- Ma trận T1 (khoảng 8 000 trial LLM) trên emulator/Cuttlefish; đo độ trễ và bộ nhớ trên Pixel thật.
- Họ tấn công kịch bản (3, 4, TOCTOU của 7) với $n \geq 10^4$, không LLM.
- Ghi *mọi* loại trừ theo nhóm (agent crash, platform crash, network outage, user cancel, định danh không hỗ trợ); chặn trước chính sách outlier.
- Xuất raw secret-free: `trials.jsonl`, `summary.json`, `metrics.csv`, `provenance.json` mở rộng (hash của git, AOSP, build, kernel, APK, mô hình, prompt, chính sách).
- Sinh bảng/hình *chỉ* từ các tệp raw này.

Sản phẩm bàn giao:

Bộ chạy tái lập; raw outcomes; provenance đầy đủ; báo cáo killer experiment.

Tiêu chí nghiệm thu (Definition of Done):

(a) giá trị thô vắng trong mọi artifact; (b) redirect bị từ chối; (c) login hợp lệ thành công; mọi loại trừ được ghi và giải thích; bảng/hình sinh từ raw.

22 Giai đoạn 8 — Mở rộng bậc T2 (chỉ khi G2 qua; W9–W12)

Mục tiêu: tăng độ phủ nếu còn thời gian; không được làm trễ T1.

- Adapter clipboard nhạy cảm (chỉ clip được đánh dấu nhạy cảm; thay nội dung clip hướng tác nhân bằng projection công khai; giữ hành vi dán thường cho người dùng).
- App C (địa chỉ + điện thoại); B2 và so sánh với cơ chế che sẵn có của nền tảng.
- Đo năng lượng (Perfetto/Batterystats), tác nhân thứ hai, hồi quy tác vụ chung.
- Kiểm thử WebView ở chế độ fail-closed: đo tỉ lệ từ chối sai và ghi "unsupported" rõ ràng.

23 Giai đoạn 9 — Hạng mục bậc T3 và hướng phát triển (ngoài lịch nộp 04/12/2026)

Các hạng mục sau không nằm trong lịch tới 04/12/2026 và *không* được nêu như đóng góp nếu chưa làm: Credential Manager (chỉ đúc sau khi người dùng chọn credential cho app/origin yêu cầu), ví/wallet và xác nhận mỗi lần dùng (app D), Protected Confirmation, xác thực WebView origin (node→frame→origin; fail-closed nếu không đủ), kho bền vững + nhật ký sự cố + đa người dùng/sao lưu, khóa StrongBox/Knox, thiết bị OEM. Đây là nội dung tự nhiên cho bản mở rộng/Plan B.

24 Giai đoạn 10 — Phân tích và viết bài (W5–W7 cho bản nộp; W9–W15 cho bản full)

Mục tiêu: bản nộp 04/12 (Emerging Ideas) dựa trên bằng chứng sơ bộ; bản full mới cần số đo thật.

24.1 Công việc

- Với bản nộp 04/12: trình bày *bằng chứng sơ bộ* từ mô hình tham chiếu + tái hiện tấn công; **ghi rõ** là mô phỏng/tiền thực nghiệm, *không* phải kết quả hệ thống AOSP.
- **Rút gọn bản thảo 53 trang về giới hạn MobiSys: 12 trang thân bài** (đã xác nhận trong CFP). Chuyển chứng minh chi tiết và thí nghiệm bổ sung sang phụ lục (không tính trang; reviewer có thể bỏ qua).

- Báo cáo kết quả nhị phân dạng *0 of n* + khoảng tin cậy Wilson; *không* tuyên bố bằng chứng tuyệt đối từ zero.
- Độ trễ: trung vị, phân vị đuôi, bootstrap interval (theo cụm), phân phối đầy đủ.
- So sánh cặp: McNemar chính xác / permutation cặp; RQ1–RQ4 là primary.
- Thay *mọi* dấu gạch/TODO trong bảng kết quả, tham số, ablation, tương thích bằng giá trị đo, cài đặt đóng băng, "unsupported" hoặc "not applicable".
- Thay/gỡ tọa độ PGFPlots khái niệm; không biến xu hướng kỳ vọng thành kết quả khi chưa có dữ liệu thô + CI.
- Kiểm giả định định lý bằng oracle thực nghiệm; nêu rõ giả định nào còn mang tính kiến trúc.
- Cập nhật phần liên quan/điểm mới; chạy lại tìm kiếm tài liệu ngay trước nộp; xác minh toàn văn các so sánh mới chỉ có abstract.
- Chuyển bản thảo sang `\documentclass[sigconf,10pt]{acmart}` (mẫu MobiSys; US Letter, double-column); **single-blind từ 2026 nên phải ghi tên/cơ quan tác giả** (không ẩn danh); điền tuyên bố lợi ích/dữ liệu/đạo đức (canary tổng hợp ⇒ không human subjects); tuân thủ chính sách AI/LLM của venue.

Sản phẩm bàn giao:

Bản thảo đúng giới hạn trang với số đo thật; tìm kiếm tài liệu cập nhật.

Tiêu chí nghiệm thu (Definition of Done):

Không còn TODO/gạch trong bảng; mọi hình/bảng tái tạo từ raw; metadata cuối đã điền; checker nộp bài xanh.

25 Giai đoạn 11 — Đóng gói artifact và nộp bài (W6–W7; hạn 04/12/2026)

Mục tiêu: đóng gói artifact tái lập + nộp đúng hạn.

- Phát hành: mã mô hình tham chiếu, mô phỏng, build manifest, hash app/tác nhân/mô hình (nếu có), prompt, seed, canary monitor, schema raw-data, script phân tích, license artifact.
- Artifact evaluation của MobiSys là *tùy chọn* (tiến hành sau khi được nhận); nếu chọn, cung cấp implementations/test/data. Tách test cần thiết bị khỏi test emulator.
- Một lệnh build/run + smoke test canary cô lập offline; CI Python 3.11/3.12 (pipeline thiết bị kích hoạt thủ công).
- Rà soát `TODO_before_submission.md` lần cuối; kiểm format/accessibility/plagiarism/self-overlap.
- Nộp *trước* hạn vài ngày (mục tiêu $\leq$ 01/12/2026; hạn 04/12/2026; đăng ký abstract 27/11/2026).

Sản phẩm bàn giao:

Artifact release + hướng dẫn một lệnh + CI; bài đã nộp.

Tiêu chí nghiệm thu (Definition of Done):

Người đánh giá độc lập chuyển B0–B3 không đổi task text, xem được quyết định audit, xác minh cô lập oracle.

Phần III — Lộ trình, tiêu chí, rủi ro

26 Lộ trình thời gian

26.1 Plan A — MobiSys 2027 Emerging Ideas (nộp 04/12/2026)

Tính từ thứ Hai 12/10/2026 (W1) tới hạn nộp 04/12/2026 (W8): 8 tuần. Đây là bản *Emerging Ideas* (chưa đòi đánh giá đầy đủ); **Track B là đường găng**, Track A (AOSP) chạy song song phục vụ Plan B. Các cổng ở Mục 26.3 là nơi quyết định cắt phạm vi.

Tuần	Ngày	Track A (AOSP — Plan B)	Track B (đường găng bản nộp)	Cổng
1–2	12/10–25/10	Khởi động `repo sync` AOSP (song song, không gấp)	Dựng môi trường dev (venv 3.12, `ruff`/`mypy`); **tái hiện B0** trên emulator/stock; monitor bản đầu; chốt D0–D6	G0
3–5	26/10–15/11	Build ảnh AOSP trên emulator/Cuttlefish (Plan B)	Tái hiện đầy đủ họ tấn công (1, 3, 4, 7); bằng chứng sơ bộ từ mô hình tham chiếu; khung bản thảo (problem/threat/design)	G1
6–7	16/11–27/11	—	Viết bản thảo; chuyển `acmart` sigconf; ghi tên tác giả; **đăng ký abstract (27/11)**	G2
7–8	28/11–04/12	—	Đóng băng bản thảo 12 trang + phụ lục; kiểm format; **nộp 04/12**	G3 / Nộp

26.2 Plan B — bản full có số đo thật, kỳ nộp kế tiếp

Bản full (T1 AOSP trên emulator/Cuttlefish + killer experiment + ma trận T1 + số đo thiết bị thật nếu có) nhắm kỳ nộp kế tiếp: MobiSys 2028 / USENIX Security / NDSS / IEEE S&P. Hạn các kỳ này *chưa* được kiểm tra trong lần rà soát này — tra cứu khi lập lịch. Lưu ý: bài Emerging Ideas đang dưới review tới 06/03/2027, nên bản full không được chồng lấn nội dung (tránh double submission) cho tới khi có quyết định.

26.3 Các cổng quyết định

Xem bảng cổng G0–G3 và Nộp ở Mục 13.3 (Phần II). Mỗi cổng phải có một biên bản ngắn: kết quả đo, quyết định, thay đổi phạm vi.

27 Bảng tiêu chí hoàn thành tổng thể (Definition of Done)

Hạng mục	Tiêu chí	Bằng chứng
Cơ chế AOSP (T1)	Dịch vụ + adapter SMS + resolver + redactor chạy trên Pixel userdebug và emulator; tác nhân chạy ở UID shell không root, SELinux enforcing	Patch set, build manifest, log kiểm thử
An ninh (T1)	Không lộ giá trị trong mọi artifact; chuyển hướng bị từ chối; handle bất khả giả mạo; single-use/TTL đúng	Báo cáo killer + audit; cận Wilson theo cỡ mẫu
Tiện ích (T1)	Login/2FA hoàn thành với overhead chấp nhận được; U có điều kiện trên B0	Bảng U + độ trễ
Tái lập	Một lệnh build/run + smoke test canary offline	Hướng dẫn artifact + CI
Trung thực kết quả	Không còn placeholder trong bảng/hình; mọi số có CI; hạng mục T3 chưa làm nằm trong Giới hạn	Bản thảo cuối + raw data
Tài liệu	Spec, traceability, gap audit, hợp đồng adapter, biên bản các cổng, quyết định D0–D5	Thư mục `audits/`, `docs/`

28 Rủi ro và phương án giảm thiểu

Rủi ro	Tác động	Phương án
Lịch 8 tuần (MobiSys) quá ngắn cho full paper	Không kịp có số đo thật	Nộp **Emerging Ideas** (không đòi evaluation đầy đủ); bằng chứng sơ bộ + kế hoạch; bản full để Plan B
Không có thiết bị thật (Pixel)	Không lấy được độ trễ/năng lượng + keystore phần cứng	D6: emulator/Cuttlefish chạy ảnh AOSP tự build; các mục này chuyển sang Giới hạn/future work
Tác nhân điều khiển qua ADB chạy UID shell/root	Redactor/resolver không nhận ra principal; kết quả vô nghĩa	Quyết định D3; đánh giá với `adb unroot`, SELinux enforcing; thử ba kiểu principal
Bề mặt nguồn (thông báo, ứng dụng SMS) lộ OTP trước `ENTER`	Tác nhân đọc OTP từ app SMS, phá luận điểm chính	Che theo cửa sổ nguồn; che theo giá trị chỉ là lớp bổ sung; fail-closed; kiểm trong họ tấn công 1
Redaction ảnh chụp trước khi ánh xạ khó làm	Trễ G1/G2	Phương án lùi: che theo cửa sổ/bỏ frame; ghi giới hạn
Nút cổ chai Track A (một kỹ sư AOSP)	Trễ đường AOSP (không chặn bản nộp)	AOSP chỉ là Plan B; đường găng MobiSys là Track B (B0/tấn công/monitor) không cần AOSP
Thông lượng thiết bị và chi phí LLM	Không chạy xong ma trận	Emulator/Cuttlefish cho trial hàng loạt; tách tấn công kịch bản khỏi LLM; trần chi phí API; ma trận T1 $\approx 8\,000$ trial
"2602.10139 đã làm placeholder rồi"	Reviewer nghi ngờ điểm mới	Nhấn mạnh nguồn đúc, bất khả giả mạo, ràng buộc đích, vòng đời, đánh giá OTP, mô hình đe dọa; đo B1
"Đây là Autofill" / "Android đã có FLAG_SECURE, che mật khẩu"	Nhầm lẫn cơ chế	Chứng minh Autofill do app khởi xướng/tin app; B2 mở rộng đo phần cơ chế nền tảng bao phủ được
Tái lộ sau khi nhập	Điểm mới bị phá	Redactor phải hoạt động và được đo; fail-closed; harness đọc lại mọi kênh

Rủi ro	Tác động	Phương án
Binding WebView origin quá thô	Ràng buộc yếu cho web	T1 chỉ view gốc; WebView fail-closed, ghi "unsupported"; hạng mục T3
Injection gây hành động hợp lệ nhưng không mong muốn	Giới hạn tuyên bố	Phạm vi hóa tuyên bố; đo "same-destination misuse" (họ tấn công 6) và báo cáo kết quả khác 0
Bản thảo 53 trang vượt giới hạn trang	Bị từ chối vì định dạng	Rút về 12 trang thân bài (MobiSys); chứng minh/thí nghiệm bổ sung chuyển phụ lục
Chính sách venue (single-blind, đạo đức, AI/LLM, artifact)	Từ chối hoặc phải sửa gấp	Đọc CFP chính thức ở G0; ghi tên tác giả; gán nhãn LLM; tuân thủ ngay từ bản nháp
Không có dữ liệu/kết quả thật	Bài bị coi là chưa hoàn chỉnh	Nộp **Emerging Ideas** (ưu tiên insight); bằng chứng sơ bộ ghi rõ là mô phỏng; *không* biến placeholder thành kết quả
Tài nguyên (thiết bị/license/mô hình) chậm	Trượt tiến độ	Cấp phát song song từ tuần 1; emulator cho phần không cần phần cứng

29 Theo dõi tiến độ

Checklist theo dõi tiến độ (trạng thái từng hạng mục, cổng G0–G3, bảng rủi ro và hành động tiếp theo) đã được tách sang tài liệu riêng `MASTER_CHECKLIST.pdf` để tránh trùng lặp với phần mô tả và hướng dẫn trong tài liệu này. Hãy dùng tài liệu đó làm nguồn duy nhất để biết "đã làm đến đâu" và "bước tiếp theo là gì".

30 Phụ lục A — Cấu trúc thư mục và ánh xạ thành phần

Đường dẫn	Vai trò
`instructions/`	Tài liệu này (mô tả + hướng dẫn triển khai)
`sections/`	Các chương bản thảo (`introduction`, `system_model`, `methodology`, `algorithm`, `theory`, `experiments`, …)
`appendix/`	`proofs.tex`, `additional_experiments.tex`
`figures/`	TikZ/PGFPlots (kiến trúc, pipeline, threat/privacy, kết quả khái niệm)
`SecretHandle/src/secrethandle/`	Gói mô hình tham chiếu (`core`, `crypto`, `policy`, `store`, `metrics`, `statistics`, `adapters`, …)
`SecretHandle/configs/`	Cấu hình tổng hợp và template EC2
`SecretHandle/docs/`	Schema dữ liệu, adapter, implementation notes, EC2, tái lập
`SecretHandle/tests/`	Bộ pytest offline
`SecretHandle/audits/`	Ánh xạ paper–mã, audit math/result/gap
`audits/` (gốc)	Audit citation/figure/math/reviewer/notation (cấp bản thảo)
`output/`, `tmp/`, `figure_previews/`	Bản PDF, ảnh QA, preview hình

31 Phụ lục B — Lệnh thao tác nhanh

Kiểm thử mô hình tham chiếu:

```
python3 -m pytest -q
```

Chạy demo tổng hợp và sinh hình:

```
python3 scripts/run_synthetic_demo.py \
  --config configs/synthetic_demo.yaml
python3 scripts/make_plots.py \
  --input outputs/synthetic_demo/metrics.csv \
  --output outputs/synthetic_demo/plots
python3 scripts/run_experiment.py \
  --config configs/synthetic_demo.yaml --stage killer
```

Đưa thiết bị về trạng thái không root trước khi chạy tác nhân (đánh giá):

```
adb unroot
adb shell getenforce     # kỳ vọng: Enforcing
```

Biên dịch bản thảo (từ thư mục gốc dự án, nơi có `main.tex`):

```
pdflatex main
bibtex main
pdflatex main
pdflatex main
```

Biên dịch tài liệu hướng dẫn này:

```
xelatex SecretHandle_HuongDan.tex
xelatex SecretHandle_HuongDan.tex
```

32 Phụ lục C — Sơ đồ phụ thuộc công việc

Đường nộp 04/12/2026 (MobiSys Emerging Ideas):

Track B (tuần 1): dựng môi trường dev + **tái hiện B0** trên emulator/stock + monitor bản đầu
↓ (G0)
Tái hiện đầy đủ họ tấn công (1, 3, 4, 7) + bằng chứng sơ bộ từ mô hình tham chiếu (G1)
↓
Khung & viết bản thảo (problem/threat/design) → **đăng ký abstract** (G2)
↓
Đóng băng bản thảo 12 trang (`acmart` sigconf, ghi tên tác giả) + phụ lục (G3) → **Nộp 04/12**

Đường Plan B (bản full, kỳ sau):

Track A: build ảnh AOSP trên emulator/Cuttlefish → dịch vụ `system_server` (mint, resolver, target token, private input, redaction, audit)
↓
B3 chạy được → killer experiment đầy đủ + ma trận T1 + stress kịch bản (raw data)
↓
Số đo thiết bị thật (nếu có Pixel) → phân tích thống kê → bản full → nộp MobiSys 2028 / USENIX / NDSS / IEEE S&P

33 Phụ lục D — Nhật ký thay đổi so với phiên bản 1

Thay đổi	Lý do
Đổi venue (v2→v3, 2026-10-17): MobiSys 2027 — Emerging Ideas thay cho USENIX Security '27 Cycle 2	CFP MobiSys đã công bố (abstract 27/11/2026, nộp 04/12/2026); còn 7 tuần không đủ cho full paper; nhóm không có Pixel nên thêm D6 (emulator/Cuttlefish); Emerging Ideas không đòi evaluation đầy đủ, phù hợp trạng thái hiện tại
Đổi mục tiêu nộp: USENIX Security '27 Cycle 2 (MVP) + Plan B	Lịch 21 tuần bắt đầu từ 10/2026 không khớp hạn MobiSys 2027 (ước tính đầu 12/2026, chưa công bố chính thức); hạn 26/01/2027 của USENIX Security Cycle 2 là mốc khả dĩ duy nhất cho một bài có số đo thật
Thêm hệ thống Tier T1/T2/T3	Phạm vi gốc quá rộng cho 2 kỹ sư (4 nguồn, 4 app, WebView, Protected Confirmation, kho bền vững)
Đưa killer experiment lên W5 (walking skeleton)	Rủi ro lớn nhất lộ ra khi chạy đầu-cuối; bản gốc để tới W13–16 sau khi xây kho bền vững
Thêm các cổng quyết định (nay G0–G3 cho MobiSys) với điều kiện qua và phương án lùi	Bản gốc chỉ có mốc sản phẩm, không có điểm cắt
Tách Track A / Track B; B0, B1, tái hiện tấn công chạy từ tuần 1	B0/B1 không cần AOSP đã sửa; Track B không nên chờ Track A
Thêm định nghĩa principal tác nhân (UID app / UID shell / serialize) và đánh giá ở `adb unroot`	Nhiều tác nhân mã nguồn mở điều khiển qua ADB; không định nghĩa thì redactor không biết chặn ai
Thêm bề mặt nguồn và chính sách che theo cửa sổ	Giá trị OTP hiện diện trong thông báo/ứng dụng SMS trước `ENTER`; che theo giá trị có thể sót
Chốt D1–D5 (MAC, lưu trữ, principal, u, WebView)	Các quyết định làm đổi phạm vi và chứng minh; D1: MAC có thể thừa khi kho nằm trong TCB
Sửa mâu thuẫn giảm u giữa Algorithm 2 và mục Vòng đời	Hai chỗ nói khác nhau về thời điểm giảm u
Thêm cỡ mẫu (cận Wilson), ngân sách chạy, planner đối kháng kịch bản, tách tấn công kịch bản khỏi LLM	Ma trận gốc ước tính hơn 200 000 trial; cỡ mẫu cho tuyên bố "0 rò rỉ" chưa được nêu
Thêm B2 mở rộng (FLAG_SECURE, che mật khẩu, content sensitivity)	Reviewer sẽ hỏi cơ chế che sẵn có của Android đã bao phủ đến đâu
Thêm kế hoạch rút gọn bản thảo 53 trang, chính sách AI/LLM, đạo đức, artifact	Giới hạn trang và chính sách venue
Sửa kỹ thuật LaTeX	Phông Times New Roman/Arial/Consolas không có trên Linux/Overleaf nên bản gốc không biên dịch được (thêm phương án thay thế); `\subsubsection` đứng ngay dưới `\section` tạo số kiểu 9.0.1 (đổi thành `\subsection`); bảng ở trang bìa không xuống dòng; thêm tên mục lục tiếng Việt; sửa lỗi gõ "đượcload"

Ghi chú cuối: Tài liệu phản ánh trạng thái kho mã tại thời điểm lập. Hạn nộp và chính sách venue lấy từ

CFP chính thức MobiSys 2027 (tra cứu 17/10/2026) và phải được xác minh lại trên trang chính thức. Khi có kết quả đo thật, cập nhật Phần I (khoảng trống) và Phần II (tiêu chí nghiệm thu), và luôn giữ nguyên tắc: không placeholder nào được trình bày như kết quả.